EXERCISE 2

1) Define an integer array. Declare an integer pointer and initialize it to point the first element of the array. Use pointer arithmetic to traverse the array and print the elements.

```
#include <stdio.h>
int main()
{
   int arr[50] ;
   int *p;
   int i,n;
   p = arr;
   printf("Enter n: ");
   scanf("%d",&n);
   printf("Array elements: ");
   for(i=0;i<n;i++)
   {
      scanf("%d",&arr[i]);
   }
   printf("Original array:\n");
   for(i = 0; i < n; i++)
   {
```

```
        printf("%d ", *(p + i));
    }
    for(i = 0; i < n; i++)
    {
        *(p + i) = *(p + i) + 5;
    }
    printf("\nModified array:\n");
    for(i = 0; i < n; i++)
    {
        printf("%d ", *(p + i));
    }
    return 0;
}
```

OUTPUT:

Enter n: 4

Array elements: 10

50

20

70

Original array:

10 50 20 70

Modified array:

15 55 25 75

2) Calculate string length using pointers.

```
#include <stdio.h>

int main()

{

   char str[50];

   char *p=str;

   int length = 0;

   printf("Enter a string: ");

   scanf("%[^\n]", str);

   while(*p != '\0')

   {

      length++;

      p++;

   }

   printf("Length = %d", length);

   return 0;

}
```

OUTPUT:

Enter a string: Wonderful view

3) Write a function that calculates both the sum and product of two numbers. Return these two results using pointers passed as arguments to the function.

```
#include <stdio.h>

void calculate(int a, int b, int *sum, int *prod)

{

   *sum = a + b;

   *prod = a * b;

}

int main()

{

   int a, b;

   int sum, product;

   printf("Enter two numbers: ");

   scanf("%d\n%d", &a, &b);

   calculate(a, b, &sum, &product);

   printf("Sum = %d\n", sum);

   printf("Product = %d", product);

   return 0;

}
```

OUTPUT:

Enter two numbers: 5

7

Sum = 12

Product = 35

4) Create an array of structures. Use dynamic memory allocation to allocate memory space and use a structure pointer with pointer arithmetic to iterate and access/modify members.

```
#include <stdio.h>

#include <stdlib.h>

struct Student

{

    int id;

    char name[20];

    float mark;

};

int main()

{

    struct Student *p;

    int n, i;

    printf("Enter number of students: ");

    scanf("%d", &n);
```

```
    p = (struct Student *)malloc(n * sizeof(struct Student));
    for(i = 0; i < n; i++)
    {
        printf("\nEnter details of student %d\n", i + 1);
        printf("ID: ");
        scanf("%d", &(p+i)->id);
        printf("Name: ");
        scanf("%s", (p+i)->name);
        printf("Mark: ");
        scanf("%f", &(p+i)->mark);
    }
    printf("\nStudent Details:\n");
    for(i = 0; i < n; i++)
    {
        printf("\nID = %d", (p+i)->id);
        printf("\nName = %s", (p+i)->name);
        printf("\nMark = %.2f\n", (p+i)->mark);
    }
    free(p);
    return 0;
}
```

OUTPUT:

Enter number of students: 3

Enter details of student 1

ID: 1

Name: rohi

Mark: 95

Enter details of student 2

ID: 2

Name: kavi

Mark: 96

Enter details of student 3

ID: 3

Name: prani

Mark: 97

Student Details:

ID = 1

Name = rohi

Mark = 95.00

ID = 2

Name = kavi

Mark = 96.00

ID = 3

Name = prani

Mark = 97.00

5) Define two functions with same signature. Declare a function pointer and use the pointer to call both functions.

```
#include <stdio.h>

int add(int a, int b)

{

   return a + b;

}

int multiply(int a, int b)

{

   return a * b;

}

int main()

{

   int (*fp)(int, int);

   int a = 10, b = 5;

   fp = add;
```

```
    printf("Sum = %d\n", fp(a, b));
    fp = multiply;
    printf("Product = %d", fp(a, b));
    return 0;
}
```

OUTPUT:

Sum = 15

Product = 50

6) Declare an integer, a pointer to it and a double pointer. Print the values and addresses at each level.

```
#include <stdio.h>
int main()
{
    int x = 10;
    int *p;
    int **q;
    p = &x;
    q = &p;
    printf("Value of x = %d\n", x);
    printf("Value using p = %d\n", *p);
```

```
    printf("Value using q = %d\n", **q);
    printf("Address of x = %p\n", &x);
    printf("Address stored in p = %p\n", p);
    printf("Address of p = %p\n", &p);
    printf("Address stored in q = %p\n", q);
    return 0;
}
```

OUTPUT:

Value of x = 10

Value using p = 10

Value using q = 10

Address of x = 0x7ffe4aa31e54

Address stored in p = 0x7ffe4aa31e54

Address of p = 0x7ffe4aa31e48

Address stored in q = 0x7ffe4aa31e48

7) Sort an array of floating point numbers using selection sort.

```
#include <stdio.h>
int main()
{
    float temp;
    int i, j, min,n;
```

```
float arr[50] ;
printf("Enter n:");
scanf("%d",&n);
printf("Enter floating numbers:");
for(i=0;i<n;i++)
{
   scanf("%f",&arr[i]);
}
for(i = 0; i < n-1; i++)
{
   min = i;
   for(j = i + 1; j < n; j++)
   {
      if(arr[j] < arr[min])
      {
         min = j;
      }
   }
   temp = arr[i];
   arr[i] = arr[min];
   arr[min] = temp;
```

```
    }
    printf("Sorted array:\n");
    for(i = 0; i < n; i++)
    {
        printf("%.1f ", arr[i]);
    }
    return 0;
}
```

OUTPUT:

Enter n:5

Enter floating numbers:3.4

9.8

5.1

7.5

2.9

Sorted array:

2.9 3.4 5.1 7.5 9.8

8) Define a structure student with fields such as id, name and grade. Create an array of student structures and sort them based on different fields.

```
#include <stdio.h>
#include <string.h>
```

```
struct Student
{
    int id;
    char name[20];
    char grade;
};
int main()
{
    struct Student s[5], temp;
    int n, i, j, choice;
    printf("Enter number of students: ");
    scanf("%d", &n);
    for(i = 0; i < n; i++)
    {
        printf("\nEnter student %d details:\n", i + 1);
        printf("ID: ");
        scanf("%d", &s[i].id);
        printf("Name: ");
        scanf("%s", s[i].name);
        printf("Grade: ");
        scanf(" %c", &s[i].grade);
```

```
}
printf("\nSort based on:\n");
printf("1. ID\n");
printf("2. Name\n");
printf("3. Grade\n");
printf("Enter choice: ");
scanf("%d", &choice);
for(i = 0; i < n - 1; i++)
{
   for(j = i + 1; j < n; j++)
   {
      if((choice == 1 && s[i].id > s[j].id) ||
         (choice == 2 && strcmp(s[i].name, s[j].name) > 0) ||
         (choice == 3 && s[i].grade > s[j].grade))
         {
          temp = s[i];
          s[i] = s[j];
          s[j] = temp;
         }
   }
}
```

```
    printf("\nSorted Students:\n");
    for(i = 0; i < n; i++)
    {
        printf("ID: %d NAME: %s GRADE: %c\n", s[i].id, s[i].name, s[i].grade);
    }
    return 0;
}
```

OUTPUT

Enter number of students: 2

Enter student 1 details:

ID: 3

Name: KAVI

Grade: A

Enter student 2 details:

ID: 2

Name: PRANI

Grade: A

Sort based on:

1. ID

2. Name

3. Grade

Enter choice: 2

Sorted Students:

ID: 3 NAME: KAVI GRADE: A

ID: 2 NAME: PRANI GRADE: A

9) Sort a set of negative numbers using radix sort.

```
#include <stdio.h>

int getMax(int arr[], int n)

{

   int max = arr[0];

   int i;

   for(i = 1; i < n; i++)

   {

      if(arr[i] > max)

         max = arr[i];

   }

   return max;

}

void countingSort(int arr[], int n, int exp)

{

   int output[100];
```

```
    int count[10] = {0};
    int i;
    for(i = 0; i < n; i++)
        count[(arr[i] / exp) % 10]++;
    for(i = 1; i < 10; i++)
        count[i] = count[i] + count[i - 1];
    for(i = n - 1; i >= 0; i--)
    {
        output[count[(arr[i] / exp) % 10] - 1] = arr[i];
        count[(arr[i] / exp) % 10]--;
    }
    for(i = 0; i < n; i++)
        arr[i] = output[i];
}
void radixSort(int arr[], int n)
{
    int i, j, k = 0;
    int positive[100], negative[100];
    int p = 0, q = 0;
    for(i = 0; i < n; i++)
    {
```

```
    if(arr[i] < 0)
        negative[q++] = -arr[i];
    else
        positive[p++] = arr[i];
}
if(p > 0)
{
    int max = getMax(positive, p);
    for(int exp = 1; max / exp > 0; exp *= 10)
        countingSort(positive, p, exp);
}
if(q > 0)
{
    int max = getMax(negative, q);
    for(int exp = 1; max / exp > 0; exp *= 10)
        countingSort(negative, q, exp);
}
for(i = q - 1; i >= 0; i--)
    arr[k++] = -negative[i];
for(j = 0; j < p; j++)
    arr[k++] = positive[j];
```

```
}

int main()

{

   int arr[100], n, i;

   printf("Enter number of elements: ");

   scanf("%d", &n);

   printf("Enter elements: ");

   for(i = 0; i < n; i++)

      scanf("%d", &arr[i]);

   radixSort(arr, n);

   printf("Sorted array: ");

   for(i = 0; i < n; i++)

      printf("%d ", arr[i]);

   return 0;

}
```

OUTPUT:

Enter number of elements: 3

Enter elements: 30 40 10 -3 -1

Sorted array: -3 -1 10 30 40

10) Write a C program to copy one array to another using pointers.

```
#include <stdio.h>
```

```
int main()
{
   int arr1[100], arr2[100];
   int n, i;
   int *p, *q;
   printf("Enter number of elements: ");
   scanf("%d", &n);
   printf("Enter elements of first array: ");
   for(i = 0; i < n; i++)
   {
      scanf("%d", &arr1[i]);
   }
   p = arr1;
   q = arr2;
   for(i = 0; i < n; i++)
   {
      *(q + i) = *(p + i);
   }
   printf("First array: ");
   for(i = 0; i < n; i++)
   {
```

```
        printf("%d ", *(p + i));
    }
    printf("\nCopied array: ");
    for(i = 0; i < n; i++)
    {
        printf("%d ", *(q + i));
    }
    return 0;
}
```

OUTPUT:

Enter number of elements: 4

Enter elements of first array: 1 2 3 4

First array: 1 2 3 4

Copied array: 1 2 3 4